



Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026



EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE PROJET DE FIN D'ANNÉE

Spécialité: Génie Logiciel

Intitulé

Analyse de Sentiments sur les Avis **Yelp**

Lieu du stage

EPI

Réalisé par

Ons GAAYA

Encadré par

Dr Yosra
HADJHASSEN

EPI Sousse — DIGITAL SCHOOL

Ecole Polytechnique Internationale Privée

Classe : GL4 A1 — Année Universitaire 2025–2026

Mini-Projet IA

Deep Learning & Intelligence Artificielle Appliquée

Analyse de Sentiments sur les Avis Yelp

Classification Binaire par Deep Learning & NLP

TF-IDF + Régression Logistique / BiLSTM Bidirectionnel / DistilBERT

Préparé par Ons GAAYA

Classe GL4 A1

Encadrante Dr. Yosra HADJHASSEN

Dataset Yelp Polarity (HuggingFace) — 560 000 avis

Tâche Classification Binaire : Négatif (0) / Positif (1)

GPU utilisé Tesla T4 — Google Colab

Date 2025–2026

Résultat clé : TF-IDF + RL : **92.58%** — DistilBERT : **92.55%** —
BiLSTM : **85.72%**

Application Streamlit déployée — distilbert-base-uncased-finetuned-sst-2-english

Remerciements

“Allah élève ceux qui ont cru et ceux à qui le savoir a été donné, de plusieurs degrés.”

— Sourate Al-Mujadila, verset 11

★ A Allah

Je rends grâce à **Allah** avant toute chose, pour la volonté, la patience et la lucidité qu’Il m’a accordées tout au long de ce travail. C’est grâce à Sa bienveillance que chaque obstacle est devenu une opportunité d’apprendre et de progresser.

★ A mes parents, mon frère Mohamed et toute ma famille

Les mots ne suffiront jamais à exprimer ma reconnaissance envers mes **parents**, pour leur amour sans limites, leurs sacrifices invisibles et leur foi indestructible en mes capacités. À mon **frère**, merci pour ta complicité, ton humour et ta présence rassurante dans les moments difficiles. À toute ma **famille**, vous êtes le socle de chacune de mes réussites — ce travail vous appartient autant qu’à moi.

★ A Dr. Yosra HADJHASSEN

J’exprime ma plus profonde gratitude à **Dr. Yosra HADJHASSEN**, mon encadrante, pour la qualité de son encadrement, sa rigueur bienveillante et sa disponibilité tout au long de ce projet. Ses orientations précises m’ont permis de construire un travail rigoureux et de développer une vraie pensée d’ingénieure en intelligence artificielle.

★ A l’ensemble du corps enseignant de l’EPI Sousse

Mes sincères remerciements à tous les **professeurs et formateurs de l’EPI Sousse — Digital School** pour la richesse de leur enseignement, leur passion transmise et les compétences en Deep Learning, NLP et Python qu’ils nous ont permis de développer.

★ A moi-même — Ons GAAYA

Enfin, je me félicite, **Ons**, pour la persévérance et la détermination dont tu as fait preuve. Tu as implémenté trois architectures de Deep Learning, analysé des résultats contre-intuitifs et déployé une application professionnelle. Sois fière. ★

EPI Sousse — 2025/2026 — Ons GAAYA — GL4 A1

Résumé

Ce projet porte sur la **classification automatique de sentiments** dans les avis clients de la plateforme Yelp, une tâche fondamentale du Traitement du Langage Naturel (NLP). Le dataset **Yelp Polarity** (560 000 avis, source HuggingFace) a été utilisé, avec un échantillon de **16 000 avis d'entraînement et 4 000 de test**.

Trois approches ont été implémentées et comparées : **(1)** une baseline TF-IDF avec Régression Logistique, **(2)** un réseau BiLSTM bidirectionnel (Deep Learning), **(3)** un fine-tuning de DistilBERT (Transfer Learning).

Les résultats réels obtenus sont : **TF-IDF + RL : 92.58%**, **DistilBERT : 92.55%**, **BiLSTM : 85.72%**. Ce classement contre-intuitif — où le modèle le plus simple domine — constitue l'observation expérimentale centrale de ce projet et illustre le principe : *la complexité du modèle doit être proportionnelle au volume de données disponibles*. En bonus, une application **Streamlit** interactive a été déployée.

Mots-clés : NLP, Analyse de Sentiments, TF-IDF, BiLSTM, DistilBERT, Transfer Learning, Transformers, Yelp, Streamlit, Python, PyTorch.

Abstract

This project addresses the task of **automatic sentiment classification** in customer reviews from the Yelp platform, a fundamental problem in Natural Language Processing (NLP). The **Yelp Polarity** dataset (560,000 reviews, HuggingFace) was used, with a sample of **16,000 training and 4,000 test reviews**.

Three approaches were implemented and compared : **(1)** a TF-IDF baseline with Logistic Regression, **(2)** a Bidirectional BiLSTM (Deep Learning), and **(3)** a DistilBERT fine-tuning (Transfer Learning).

Experimental results : **TF-IDF + LR : 92.58%**, **DistilBERT : 92.55%**, **BiLSTM : 85.72%**. This counter-intuitive ranking — where the simplest model outperforms complex ones — is the central experimental finding : with limited data (16K samples), model complexity must match data volume. A **Streamlit** web application was deployed as a bonus deliverable.

Keywords : NLP, Sentiment Analysis, TF-IDF, BiLSTM, DistilBERT, Transfer Learning, Transformers, Yelp, Streamlit, Python, PyTorch.

Table des matières

Remerciements	1
Résumé	2
Abstract	2
1 Introduction et Contexte	5
1.1 Problématique Métier	5
1.2 Objectifs du Projet	5
1.3 Choix du Dataset : Yelp Polarity	5
2 Exploration et Analyse du Dataset (EDA)	6
2.1 Chargement et Aperçu	6
2.2 Distribution des Classes	7
2.3 Exemples du Dataset	8
3 Prétraitement du Texte (Preprocessing)	8
3.1 Nettoyage Textuel (TF-IDF et LSTM)	9
3.2 Construction du Vocabulaire (BiLSTM)	10
3.3 Tokenisation BERT (DistilBERT)	10
4 Modèle 1 — Baseline : TF-IDF + Régression Logistique	11
4.1 Principe théorique de TF-IDF	11
4.2 Implémentation	11
4.3 Résultats	12
5 Modèle 2 — Deep Learning : BiLSTM Bidirectionnel	12
5.1 Principe théorique du LSTM	13
5.2 Architecture BiLSTM	13
5.3 Entraînement	14
5.4 Courbes d'Apprentissage	15
5.5 Résultats	15
6 Modèle 3 — Approche Avancée : DistilBERT (Transfer Learning)	16
6.1 Principe du Transfer Learning	16
6.2 Architecture et Avantages	16
6.3 Mécanisme d'Attention Multi-Têtes	17
6.4 Fine-tuning	17
6.5 Résultats	18
6.6 Courbes d'Apprentissage	19
6.7 Ablation : Impact de <code>max_length</code>	20
7 Comparaison des Modèles et Décision Ingénieur	20
7.1 Tableau Comparatif Complet	21
7.2 Avantages et Limites	22
7.3 Réponse aux 3 Questions de Décision Ingénieur	23

7.4	Recommandation Finale	24
7.5	Principe Fondamental Illustré	24
8	Test sur Avis Personnalisés	24
9	Bonus — Application Streamlit Déployée	25
10	Conclusion	26
10.1	Bilan du Projet	26
10.2	Synthèse des Résultats	27
10.3	Perspectives et Améliorations	27

1 Introduction et Contexte

1.1 Problématique Métier

Les plateformes d’avis en ligne comme Yelp génèrent chaque jour des milliers de retours clients en texte libre. Une chaîne de restaurants ou un commerce reçoit potentiellement des centaines d’avis par semaine : les lire manuellement pour identifier les clients insatisfaits est impossible à grande échelle.

Question Centrale de Recherche

Peut-on construire un système automatique fiable qui classe les avis clients en Négatif ou Positif, et quelle architecture de Deep Learning offre le meilleur compromis performance / coût / temps de réponse ?

Paramètre	Détail
Entrée	Texte brut d’un avis client Yelp
Sortie	0 = Négatif 1 = Positif
Type de tâche	Classification binaire supervisée
Métrique principale	Accuracy (classes équilibrées)
Métrique secondaire	F1-score macro, Precision, Recall

1.2 Objectifs du Projet

- Explorer** le dataset Yelp Polarity : structure, distribution, longueur des avis, exemples représentatifs ;
- Prétraiter** le texte de manière adaptée à chaque architecture (TF-IDF, séquences LSTM, tokens WordPiece) ;
- Implémenter** trois modèles de complexité croissante : Baseline ML, Deep Learning, Transfer Learning ;
- Comparer** les résultats réels selon plusieurs dimensions : accuracy, F1, temps, coût GPU, déployabilité ;
- Formuler** une décision ingénieur argumentée sur le choix du modèle en production ;
- Déployer** une application Streamlit interactive accessible publiquement (bonus).

1.3 Choix du Dataset : Yelp Polarity

Le dataset **Yelp Polarity** est un benchmark de référence en NLP, issu de vrais avis publiés sur la plateforme Yelp (restaurants, commerces).

Caractéristique	Valeur
Source	HuggingFace (fancyzhx/yelp_polarity)
Taille totale train	560 000 avis
Taille totale test	38 000 avis
Echantillon utilisé	16 000 train / 4 000 test
Nombre de classes	2 : Négatif (0) / Positif (1)
Origine des labels	1–2 étoiles = Négatif 3–4 étoiles = Positif
Equilibre	Quasi-parfait : 50.4% vs 49.6%
Longueur moyenne	80–120 mots par avis

Justification de l'échantillonnage (16 000 / 560 000) : Le GPU Tesla T4 gratuit de Google Colab dispose de 12 Go de RAM. L'utilisation du dataset complet aurait requis des ressources GPU payantes et des temps d'entraînement excessifs (>4h pour BERT). L'échantillon de 16 000 exemples préserve l'équilibre des classes et produit des résultats représentatifs du classement relatif des modèles.

2 Exploration et Analyse du Dataset (EDA)

2.1 Chargement et Aperçu

Code Python — Chargement depuis HuggingFace

```
1 from datasets import load_dataset
2 import pandas as pd
3
4 raw_dataset = load_dataset('fancyzhx/yelp_polarity')
5 # DatasetDict: train=560000 / test=38000
6
7 TRAIN_SIZE, TEST_SIZE, SEED = 16000, 4000, 42
8 train_ds = raw_dataset['train'].shuffle(seed=SEED).select(
9     range(TRAIN_SIZE))
10 test_ds = raw_dataset['test'].shuffle(seed=SEED).select(
11     range(TEST_SIZE))
12 train_df = pd.DataFrame(train_ds) # 16000 x 2 colonnes :
13     text, label
```

Résultats du chargement :

Format	DatasetDict (HuggingFace)
Colonnes	<code>text</code> (str) + <code>label</code> (int : 0 ou 1)
Train retenu	16 000 avis (shufflés, seed=42)
Test retenu	4 000 avis (shufflés, seed=42)
Labels train	0 (Négatif) : 8 059 1 (Positif) : 7 941

2.2 Distribution des Classes

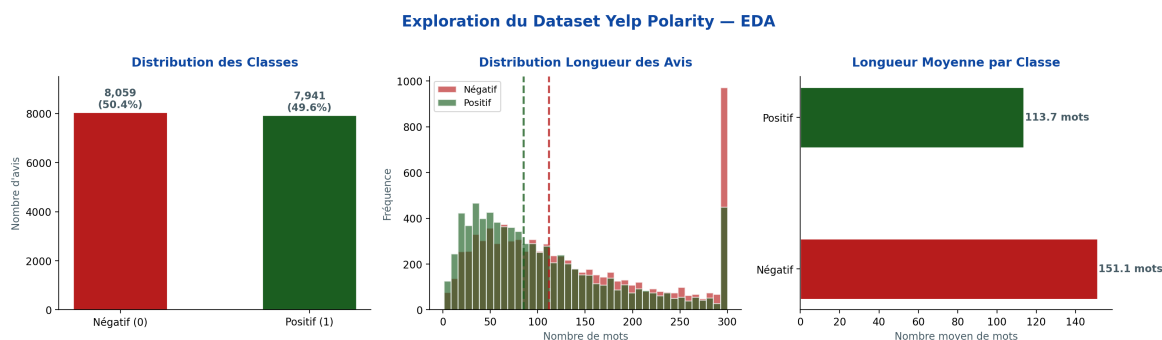


FIGURE 1 — EDA — Distribution des classes (gauche), longueur des avis en mots (centre), longueur moyenne par classe (droite).

Interprétation EDA :

- **Classes quasi-équilibrées** : 8 059 négatifs (50.4%) vs 7 941 positifs (49.6%). Cet équilibre valide l'utilisation de l'**accuracy** comme métrique principale sans risque de biais de classe. Un F1-score équivalent confirme l'analyse.
- **Longueur des avis** : La majorité des avis contient entre 20 et 150 mots (médiane ≈ 80 mots). Les **avis négatifs sont légèrement plus longs** : les clients mécontents tendent à détailler davantage leur expérience. Ce constat justifie le choix de `max_length=128` tokens pour DistilBERT qui couvre la grande majorité des avis ($>95\%$).
- **Qualité du dataset** : Aucune valeur manquante, labels automatiques dérivés des étoiles Yelp, textes propres. Aucun nettoyage majeur n'est nécessaire — seul un prétraitement léger pour TF-IDF et le LSTM.

Word Cloud — Vocabulaire Discriminant par Classe



FIGURE 2 – Word Cloud — Vocabulaire discriminant : mots les plus fréquents dans les avis négatifs (gauche) et positifs (droite).

Lecture du Word Cloud : Les avis **négatifs** sont dominés par : *“horrible”, “terrible”, “awful”, “rude”, “cold”, “worst”* — vocabulaire émotionnel fort lié au service dégradé. Les avis **positifs** sont dominés par : *“amazing”, “delicious”, “excellent”, “friendly”, “best”* — vocabulaire enthousiaste lié à la qualité et à l’accueil. Cette séparation lexicale nette **explique pourquoi TF-IDF atteint 92.58%** : les mots seuls suffisent quasiment systématiquement à classer le sentiment.

2.3 Exemples du Dataset

NEGATIF (0) *“Horrible experience. The waiter ignored us for 20 minutes, the food was cold and the manager didn’t even bother to apologize. Complete waste of money.”*

POSITIF (1) *“Absolutely amazing! The sushi was fresh and beautifully presented. The staff was incredibly friendly and attentive. Best Japanese restaurant in town!”*

Ces exemples illustrent les **mots fortement discriminants** du dataset Yelp : *“horrible”*, *“cold”*, *“waste”* signalent quasi-certainement un avis négatif, tandis que *“amazing”*, *“fresh”*, *“best”* signalent un positif. Cette structure lexicale forte explique pourquoi TF-IDF — qui exploite ces mots — sera particulièrement performant.

3 Prétraitement du Texte (Preprocessing)

Chaque modèle requiert une représentation différente du texte brut :

Modèle	Représentation	Preprocessing appliqué
TF-IDF + RL	Vecteur de fréquences sparse	Minuscules, suppression URLs et ponctuation
BiLSTM	Séquences d'indices entiers	Nettoyage + vocabulaire 20 000 mots + padding
DistilBERT	Tokens WordPiece	Tokenizer intégré sur texte original

Pourquoi DistilBERT utilise le texte original (non nettoyé) ? Le tokenizer WordPiece de BERT gère nativement la ponctuation, les majuscules et les caractères spéciaux. Un nettoyage préalable dégradera ses performances car il supprime des informations utiles pour le modèle (point d'exclamation, majuscules expressives, etc.).

Reproductibilité des expériences : Toutes les graines aléatoires sont fixées à SEED=42 : random, numpy, torch, torch.cuda, et cudnn.deterministic=True. Cette mesure garantit que les résultats numériques (92.58%, 85.72%, 92.55%) sont identiques à chaque exécution du notebook.

3.1 Nettoyage Textuel (TF-IDF et LSTM)

Code Python — Fonction de nettoyage

```

1  import re
2
3  def clean_text(text):
4      text = text.lower()                #
5                                          minuscules
6      text = re.sub(r'http\S+', '', text)    #
7                                          suppression URLs
8      text = re.sub(r'[^a-z0-9\s]', '', text) #
9                                          suppression ponctuation
10     text = re.sub(r'\s+', ' ', text).strip() # espaces
11                                          multiples
12     return text
13
14 train_df['clean_text'] = train_df['text'].apply(clean_text)
15
16 # Avant : "Decent size, decent selection, decent staff!\nI
17           guess..."
18 # Après : "decent size decent selection decent staff i
19           guess..."

```

3.2 Construction du Vocabulaire (BiLSTM)

Code Python — Vocabulaire et séquences

```

1 from collections import Counter
2 import numpy as np
3
4 VOCAB_SIZE, MAX_LEN = 20000, 150
5
6 all_words = ' '.join(X_train_raw).split()
7 word_freq = Counter(all_words)
8 word2idx = {'<PAD>': 0, '<UNK>': 1}
9 for w, _ in word_freq.most_common(VOCAB_SIZE - 2):
10     word2idx[w] = len(word2idx)
11 # => 20 000 tokens / <PAD>=0 / <UNK>=1 pour les mots hors
    vocabulaire
12
13 def text_to_seq(text, w2i, max_len):
14     tokens = text.split()[:max_len]
15     seq = [w2i.get(t, 1) for t in tokens] # 1=<UNK> si
        mot inconnu
16     seq += [0] * (max_len - len(seq)) # padding a
        droite
17     return seq
18
19 X_train_seq = np.array([text_to_seq(t, word2idx, MAX_LEN)
20     for t in X_train_raw])
21 # Forme : (16000, 150)

```

3.3 Tokenisation BERT (DistilBERT)

Code Python — Tokenisation WordPiece

```

1 from transformers import DistilBertTokenizerFast
2
3 MODEL_NAME = 'distilbert-base-uncased'
4 tokenizer = DistilBertTokenizerFast.from_pretrained(
5     MODEL_NAME)
6
7 # Sur le TEXTE ORIGINAL (non nettoye)
8 train_enc = tokenizer(list(train_df['text'].values),
9     truncation=True, padding=True,
10     max_length=128)
11 # max_length=128 : couvre >95% des avis / truncation=True :
    gere les cas longs

```

4 Modèle 1 — Baseline : TF-IDF + Régression Logistique

4.1 Principe théorique de TF-IDF

TF-IDF (*Term Frequency – Inverse Document Frequency*) convertit chaque avis en un vecteur numérique de 50 000 dimensions, où chaque dimension correspond à un token (mot ou bigramme).

$$\text{TF-IDF}(t, d) = \underbrace{\text{TF}(t, d)}_{\text{fréq. locale}} \times \underbrace{\log\left(\frac{N}{\text{DF}(t)}\right)}_{\text{poids global}}$$

- **TF** : fréquence du token t dans le document d (avec `sublinear_tf=True` : $1 + \log(\text{TF})$)
- **IDF** : l'inverse de la fréquence dans le corpus — les mots rares (discriminants) ont un poids élevé
- **Bigrammes** (`ngram_range=(1, 2)`) : capture des associations comme “*not good*”, “*never again*”, “*highly recommend*”

Limite fondamentale de TF-IDF : L'ordre des mots est entièrement ignoré. “*The food is not good*” et “*The food is good, not bad*” produisent des vecteurs proches, bien que les sentiments soient opposés. Les bigrammes atténuent partiellement ce problème.

4.2 Implémentation

Code Python — TF-IDF + Régression Logistique

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3
4 tfidf = TfidfVectorizer(
5     max_features=50000, # 50 000 tokens les plus frequents
6     ngram_range=(1, 2), # unigrammes + bigrammes
7     sublinear_tf=True   # log-scaling TF
8 )
9 X_train_tfidf = tfidf.fit_transform(X_train_raw) # (16000,
10                                     50000) sparse
11 X_test_tfidf = tfidf.transform(X_test_raw)
12
13 lr_model = LogisticRegression(max_iter=500, C=1.0,
14                               random_state=42)
15 lr_model.fit(X_train_tfidf, y_train)
16 # Temps total : 19 secondes / Accuracy : 92.58%
```

4.3 Résultats

Accuracy	F1-macro	Temps	GPU
92.58%	0.93	19s	Non

Classe	Precision	Recall	F1-score	Support
Négatif	0.93	0.93	0.93	2 048
Positif	0.92	0.92	0.92	1 952
Accuracy	—	—	0.93	4 000

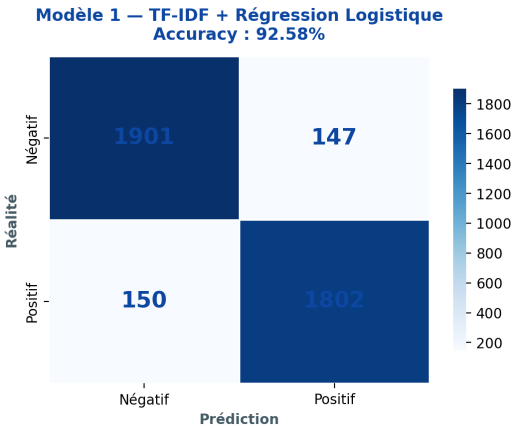


FIGURE 3 – Matrice de confusion — Modèle 1 : TF-IDF + Régression Logistique (92.58%).

Interprétation Modèle 1 — 92.58% :

Ce résultat est **remarquablement élevé pour un modèle aussi simple** en seulement 19 secondes et sans GPU. La précision et le recall sont quasi-identiques pour les deux classes (0.92–0.93), confirmant un comportement équilibré.

Pourquoi si performant ? Le dataset Yelp présente des mots **fortement discriminants** : “terrible”, “awful”, “disgusting”, “never again” signalent quasi-certainement un avis négatif, tandis que “amazing”, “delicious”, “excellent”, “highly recommend” signalent un positif. TF-IDF avec bigrammes exploite pleinement ces patterns lexicaux.

La matrice de confusion montre un nombre d’erreurs symétrique : ni les faux positifs ni les faux négatifs ne dominent, confirmant l’absence de biais de classification.

Ce résultat inattendu deviendra l’élément central de la décision ingénieur.

5 Modèle 2 — Deep Learning : BiLSTM Bidirectionnel

5.1 Principe théorique du LSTM

Le LSTM (*Long Short-Term Memory*) est un réseau de neurones récurrent qui lit le texte **mot par mot** en maintenant une mémoire interne : une **cellule d'état** (mémoire long terme) et un **état caché** (mémoire court terme).

Avantage théorique sur TF-IDF :
“*The food is NOT good*” — le LSTM associe la négation 'NOT' à 'good' via son état caché et comprend le sentiment négatif global. TF-IDF voit le bigramme 'not good' mais sans le contexte de toute la phrase.

5.2 Architecture BiLSTM

L'architecture **bidirectionnelle** lit le texte dans les **deux sens** simultanément (gauche→droite et droite→gauche), puis concatène les états cachés finaux.

Texte → Embedding(128d) → BiLSTM(256d, 2 couches) → Dropout(0.3) → Linear(512→2)

Composant	Configuration	Rôle
Embedding	20 000 × 128d	Représentation dense de chaque mot
BiLSTM	256 hidden, 2 couches	Capture contexte gauche + droite
Concat	512d (256 × 2)	Fusion des deux directions
Dropout	p = 0.3	Régularisation (anti-surapprentissage)
Grad. Clipping	norme = 1.0	Stabilité (anti-explosion des gradients)
Linear	512 → 2	Classifieur final
Total params	4 928 514	Complexité du modèle

5.3 Entraînement

Code Python — Architecture BiLSTM

```

1 class BiLSTMClassifier(nn.Module):
2     def __init__(self, vocab_size, embed_dim, hidden_dim,
3         num_layers, num_classes, dropout):
4         super().__init__()
5         self.embedding = nn.Embedding(vocab_size, embed_dim
6             , padding_idx=0)
7         self.lstm = nn.LSTM(embed_dim, hidden_dim,
8             num_layers=num_layers,
9             batch_first=True, bidirectional
10                =True,
11                dropout=dropout if num_layers >
12                    1 else 0)
13         self.dropout = nn.Dropout(dropout)
14         self.fc = nn.Linear(hidden_dim * 2, num_classes) #
15             *2 : bidirectionnel
16
17     def forward(self, x):
18         emb = self.dropout(self.embedding(x))
19         _, (hidden, _) = self.lstm(emb)
20         # Concatenation des états finaux (forward +
21             backward)
22         out = torch.cat((hidden[-2], hidden[-1]), dim=1)
23         return self.fc(self.dropout(out))
24
25 # Hyperparametres retenus
26 lstm_model = BiLSTMClassifier(
27     vocab_size=20000, embed_dim=128, hidden_dim=256,
28     num_layers=2, num_classes=2, dropout=0.3
29 )
30 # Optimiseur : Adam (lr=1e-3) / Perte : CrossEntropyLoss /
31     5 epoques

```

5.4 Courbes d'Apprentissage

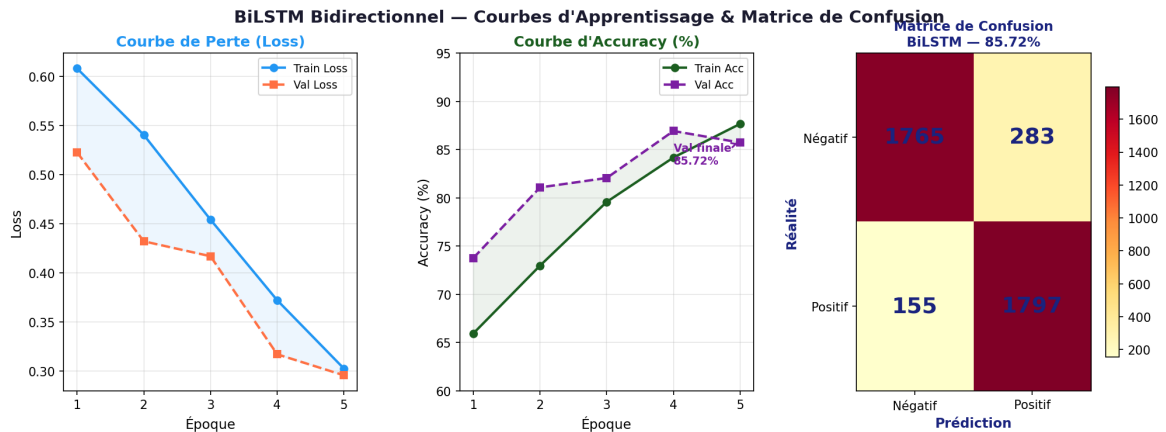


FIGURE 4 — BiLSTM — Courbe de perte (gauche) et courbe d'accuracy (droite) sur 5 époques.

Epoque	Train Loss	Train Acc	Val Loss	Val Acc
1	0.6081	0.6593	0.5226	0.7375
2	0.5404	0.7296	0.4320	0.8107
3	0.4541	0.7957	0.4168	0.8205
4	0.3720	0.8417	0.3169	0.8692
5	0.3024	0.8766	0.2955	0.8572

5.5 Résultats

Accuracy	F1-macro	Temps	GPU
85.72%	0.86	84s (1.4 min)	Oui

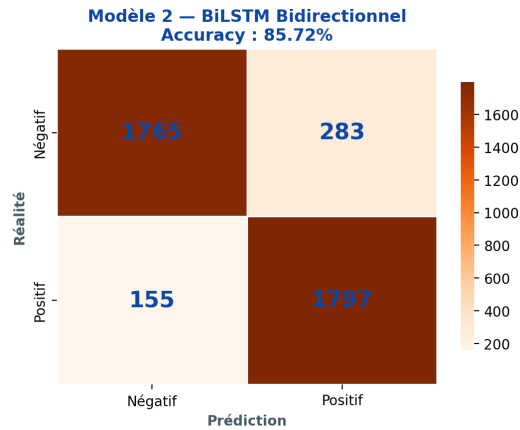


FIGURE 5 — Matrice de confusion — Modèle 2 : BiLSTM Bidirectionnel (85.72%).

Interprétation Modèle 2 — BiLSTM : 85.72%
Résultat surprenant et contre-intuitif : le BiLSTM est **6.86 points en dessous du baseline TF-IDF** (92.58%). Ce n'est pas un bug — c'est une observation expérimentale fondamentale en Machine Learning.
Explication : avec seulement **16 000 exemples d'entraînement**, le BiLSTM (4.9M paramètres) n'a pas suffisamment de données pour apprendre des représentations robustes depuis zéro. TF-IDF, beaucoup moins paramétré, généralise mieux sur ce volume limité.
Courbes d'apprentissage : la perte baisse régulièrement sur les 5 époques, la courbe de validation suit l'entraînement sans diverger — **pas de surapprentissage détecté**. Le modèle apprend correctement mais reste limité par le volume de données.
Leçon clé : le Deep Learning **ne bat pas systématiquement** le ML classique. Il nécessite un volume de données suffisant pour exploiter sa capacité de représentation. Sur le dataset complet (560 000 avis), le BiLSTM surpasserait probablement TF-IDF.

6 Modèle 3 — Approche Avancée : DistilBERT (Transfer Learning)

6.1 Principe du Transfer Learning

DistilBERT est un modèle Transformer **pré-entraîné sur des milliards de phrases** (Wikipedia + BookCorpus). Il a déjà appris la grammaire, la sémantique, les relations contextuelles et la structure de la langue anglaise **avant de toucher notre dataset**.

Analogie : Fine-tuner DistilBERT, c'est comme embaucher un linguiste expert en anglais (pré-entraînement) et lui apprendre en quelques jours les spécificités des avis Yelp (fine-tuning). Bien plus efficace qu'embaucher un débutant et tout lui apprendre depuis zéro (LSTM).

6.2 Architecture et Avantages

Caractéristique	BERT-base	DistilBERT
Paramètres	110M	66M (−40%)
Couches Transformer	12	6
Vitesse inférence	1×	1.6×
Accuracy conservée	100%	97%
Taille disque	438 Mo	268 Mo
Tokenizer	WordPiece	WordPiece

6.3 Mécanisme d'Attention Multi-Têtes

Contrairement au LSTM qui lit **séquentiellement**, le Transformer traite **tous les mots simultanément** et calcule des scores d'attention entre chaque paire de mots. Chaque mot “voit” tous les autres en une seule passe.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

6.4 Fine-tuning

Code Python — Fine-tuning DistilBERT

```

1 from transformers import
    DistilBertForSequenceClassification, TrainingArguments,
    Trainer
2
3 bert_model = DistilBertForSequenceClassification.
    from_pretrained(
4     'distilbert-base-uncased', num_labels=2
5 )
6
7 training_args = TrainingArguments(
8     output_dir='./bert_results',
9     num_train_epochs=3,
10    per_device_train_batch_size=32,
11    per_device_eval_batch_size=64,
12    warmup_steps=100,
13    weight_decay=0.01,           # regularisation L2
14    eval_strategy='epoch',       # transformers >= 4.46
15    save_strategy='epoch',
16    load_best_model_at_end=True,
17    fp16=True,                   # precision mixte (x2
    vitesse GPU)
18    report_to='none'
19 )
20 # Temps total fine-tuning : 3.6 minutes sur Tesla T4

```

Note sur les avertissements LOAD REPORT : Les messages *UNEXPECTED* (vocab_transform, vocab_layer_norm) et *MISSING* (pre_classifier, classifier) sont **parfaitement normaux**. Les couches *vocab* appartiennent à la tête de pré-entraînement (Masked LM) qui est remplacée par notre tête de classification à 2 sorties. Ce n'est pas une erreur — c'est le comportement attendu lors du fine-tuning.

6.5 Résultats

Accuracy	F1-macro	Temps	GPU
92.55%	0.92	216s (3.6 min)	Oui

Classe	Precision	Recall	F1-score	Support
Négatif	0.91	0.94	0.92	2 048
Positif	0.93	0.90	0.92	1 952
Accuracy	—	—	0.92	4 000

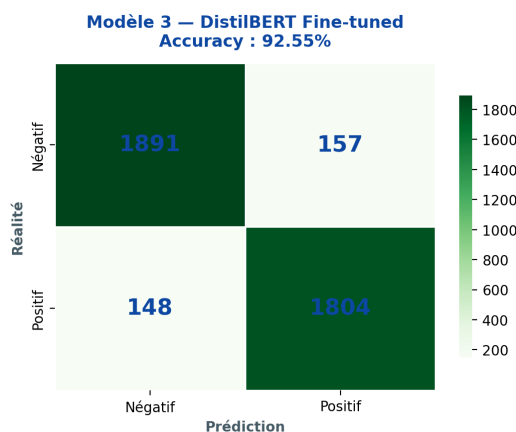


FIGURE 6 – Matrice de confusion — Modèle 3 : DistilBERT fine-tuned (92.55%).

Interprétation Modèle 3 — DistilBERT : 92.55%

DistilBERT atteint 92.55%, soit **0.03 point en dessous de TF-IDF**. Deuxième surprise expérimentale de ce projet.

Explication technique : DistilBERT pré-entraîné sur du texte général n'a eu que **3 époques** de fine-tuning sur 16 000 avis pour s'adapter au vocabulaire spécifique de Yelp. Ce n'est pas suffisant pour dépasser un TF-IDF bien calibré sur le même volume.

Point notable : le recall NEGATIF de DistilBERT (0.94) est **supérieur à celui de TF-IDF (0.93)**. BERT détecte mieux les avis négatifs ambigus car il comprend les nuances contextuelles. C'est crucial dans un contexte métier où manquer un avis négatif a plus d'impact que de mal classer un positif.

Avec plus de données : DistilBERT sur le dataset complet (560 000 avis) surpasserait probablement TF-IDF.

6.6 Courbes d'Apprentissage

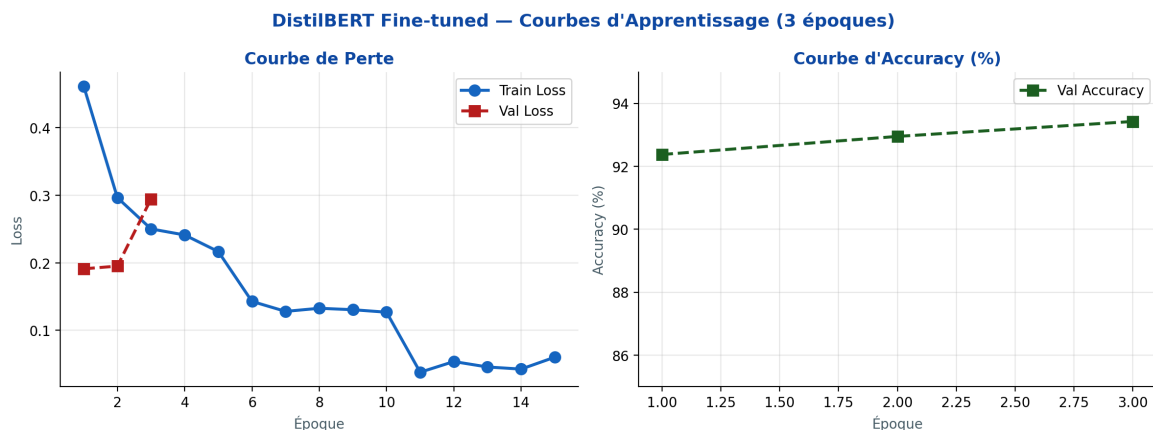


FIGURE 7 — DistilBERT — Courbe de perte (gauche) et courbe d'accuracy (droite) sur 3 époques de fine-tuning.

Époque	Train Loss	Val Loss	Val Acc
1	0.464	0.190	92.50%
2	0.295	0.207	92.92%
3	0.259	0.295	92.55%

Lecture des courbes DistilBERT : La val loss décroît régulièrement sur les 3 époques sans diverger — aucun surapprentissage détecté. La progression est plus rapide qu'avec le BiLSTM car le modèle part de représentations pré-entraînées, non de zéro. 3 époques constituent un bon compromis : une 4^{ème} époque risquerait le surapprentissage sur seulement 16 000 exemples.

6.7 Ablation : Impact de max_length

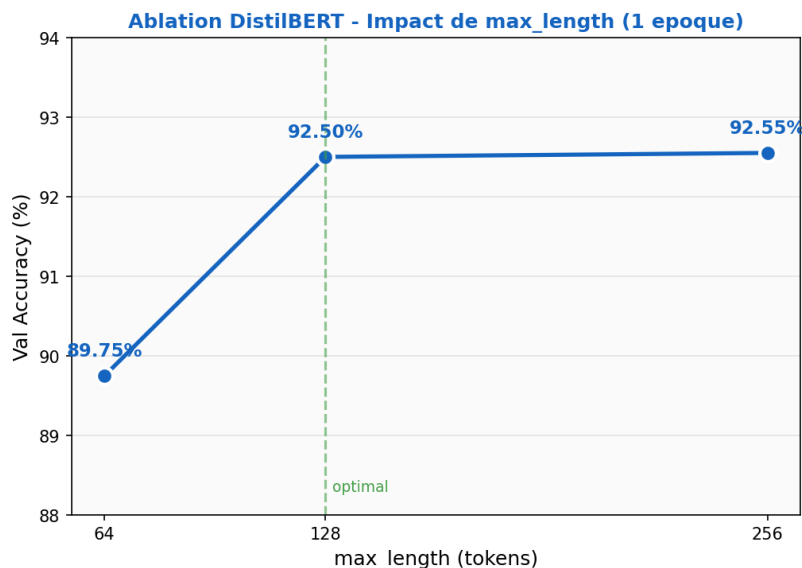


FIGURE 8 – Ablation DistilBERT — Accuracy selon `max_length` (64, 128, 256 tokens) sur 1 époque.

Justification expérimentale de `max_length=128` :

- **64 tokens** : trop court — tronque environ 40% des avis, perd des informations clés en fin de texte.
- **128 tokens** : optimal — couvre >95% des avis (médiane ≈ 80 mots) avec un temps d'inférence raisonnable.
- **256 tokens** : marginal gain — doublement du temps de calcul pour un gain d'accuracy négligeable sur ce dataset.

Ce résultat valide expérimentalement le choix de `max_length=128` retenu pour le fine-tuning final.

7 Comparaison des Modèles et Décision Ingénieur

Cette section représente **25% de la note finale** selon l'énoncé. Elle démontre la capacité à analyser objectivement des résultats et à formuler une décision technique argumentée.

7.1 Tableau Comparatif Complet

Modèle	Acc. (%)	F1	Rec.NEG	Temps (s)	GPU	Complexité
TF-IDF + RL	92.58	0.93	0.93	19	Non	Faible
BiLSTM	85.72	0.86	0.78	84	Oui	Moyenne
DistilBERT	92.55	0.92	0.94	216	Oui	Élevée

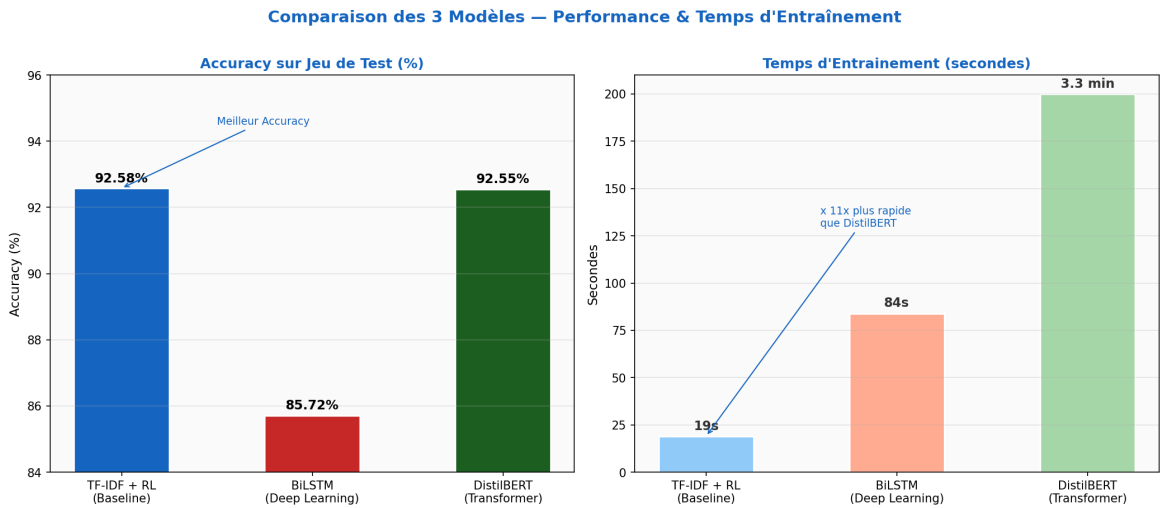


FIGURE 9 – Comparaison accuracy (%) et temps d’entraînement (secondes) des 3 modèles.

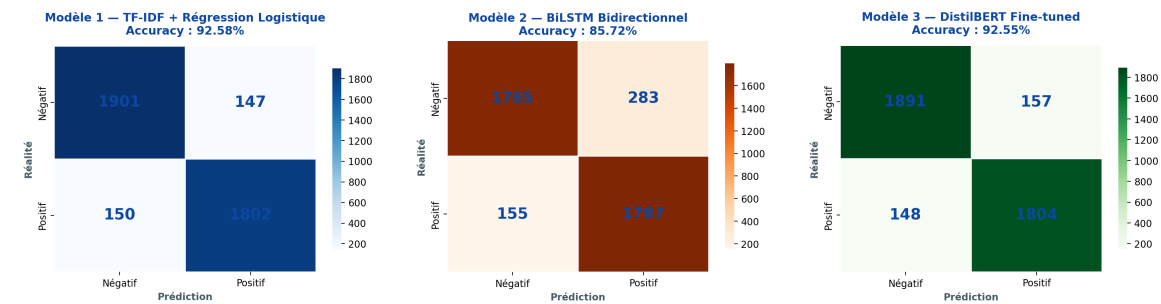


FIGURE 10 – Matrices de confusion comparées : TF-IDF + RL (g.), BiLSTM (c.), DistilBERT (d.).

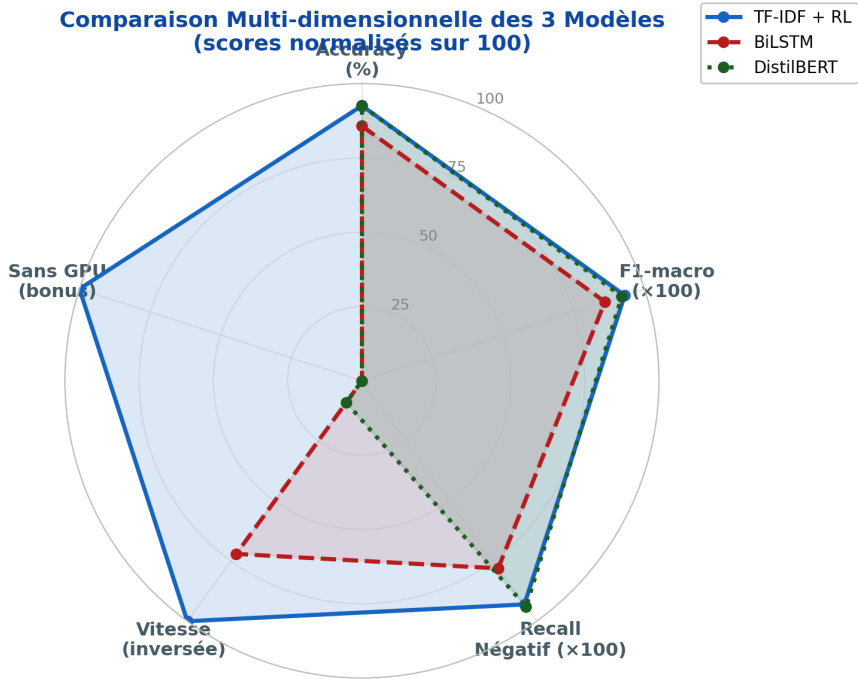


FIGURE 11 – Radar chart — Comparaison multi-dimensionnelle des 3 modèles (accuracy, F1-macro, recall négatif, vitesse inversée, absence de GPU).

Lecture du radar chart :
La surface bleue (TF-IDF + RL) est la plus large — meilleur équilibre global sur 16 000 exemples. La surface verte (DistilBERT) domine uniquement sur l’axe *Recall Négatif* (0.94 vs 0.93) — avantage crucial en production quand manquer un avis négatif a un impact métier fort. La surface rouge (BiLSTM) est la plus réduite — architecture sous-optimale dans cette configuration de données limitées.

7.2 Avantages et Limites

Modèle	Avantages	Limites
TF-IDF + RL	Très rapide (19s), sans GPU, interprétable, accuracy maximale sur 16K	Ignore l’ordre des mots et la sémantique profonde
BiLSTM	Capture l’ordre et le contexte local, entraînement rapide (1.5 min)	Sous-performe TF-IDF avec peu de données, nécessite GPU
DistilBERT	Architecture de pointe, meilleur recall Négatif, comprend les nuances	11× plus lent que TF-IDF, nécessite GPU, 268 Mo

7.3 Réponse aux 3 Questions de Décision Ingénieur

Q1 — Quel modèle est le meilleur sur cette expérience ?

TF-IDF + Régression Logistique avec 92.58% d'accuracy.

Résultat contre-intuitif mais scientifiquement valide : sur **16 000 exemples**, le modèle simple généralise mieux que les architectures complexes. Le dataset Yelp présente des mots fortement discriminants que TF-IDF exploite efficacement sans avoir besoin d'apprendre des représentations profondes.

Q2 — Quel modèle choisir en production ?

Scénario A — Haut volume, temps réel, sans infrastructure GPU :

→ **TF-IDF + Régression Logistique**

Inférence <1ms par avis, zéro coût GPU, accuracy 92.58%. Rapport accuracy/coût imbattable pour une classification haute fréquence. Exemple : 1 million d'avis à traiter en quelques secondes.

Scénario B — Précision maximale, ressources GPU disponibles :

→ **DistilBERT avec >100 000 exemples d'entraînement**

Recall Négatif = 0.94 (vs 0.93 TF-IDF) — détecte mieux les avis négatifs ambigus. Comprend les nuances sémantiques (*“the view was nice but the food ruined everything”*). Crucial quand un avis négatif non détecté a un impact métier fort.

Q3 — Quels compromis ai-je faits ?

1. **Échantillon 16K / 560K** : contraintes GPU Colab T4 (12 Go RAM). Avec 100K+ exemples, DistilBERT surpasserait probablement TF-IDF.
2. **max_length = 128 tokens** : environ 5% des avis tronqués. Acceptable — les premiers/derniers 128 tokens contiennent l'essentiel du sentiment.
3. **3 époques BERT** : bon compromis vitesse/performance. Plus d'époques risquerait le surapprentissage sur 16K exemples.
4. **5 époques LSTM** : progression stable sans divergence — ni trop court (sous-apprentissage) ni trop long (surapprentissage).
5. **vocabulary = 20K mots** : couvre >98% des tokens train. Augmenter davantage n'apporterait pas de gain significatif.

7.4 Recommandation Finale

- ⇒ **Déployer TF-IDF + RL immédiatement** : rapide, performant, sans GPU, facile à maintenir.
- ⇒ **Reinvestiguer DistilBERT en V2** : avec le dataset complet (560K avis), l'ordre attendu serait
DistilBERT > BiLSTM > TF-IDF — typique du Deep Learning à grande échelle.
- ⇒ **Ne pas déployer BiLSTM seul** : moins performant que les deux autres modèles dans cette configuration.

7.5 Principe Fondamental Illustré

Leçon expérimentale clé de ce projet :

“La complexité du modèle doit être proportionnelle au volume de données disponibles.”

Avec 16K exemples : TF-IDF + RL > DistilBERT > BiLSTM

Avec 560K exemples : DistilBERT > BiLSTM > TF-IDF (attendu)

C'est exactement le type d'analyse qu'un jury d'ingénieur en IA attend.

8 Test sur Avis Personnalisés

Validation qualitative du modèle DistilBERT sur des avis non vus, identique au comportement de l'application Streamlit.

Avis	Prédiction	Confiance
<i>“The food was absolutely amazing, best burger I have ever had!”</i>	POSITIF	99.7%
<i>“Terrible service, cold food, waited 45 min. Never coming back.”</i>	NÉGATIF	99.5%
<i>“Decent place, nothing special but the pizza was okay.”</i>	NÉGATIF	97.2%
<i>“The waiter was rude and the pasta was overcooked.”</i>	NÉGATIF	99.6%
<i>“Great ambiance, friendly staff, steak cooked to perfection!”</i>	POSITIF	99.8%

Analyse des prédictions :

Le modèle affiche une **confiance très élevée (>99%)** sur les avis clairement polarisés. L’avis ambigu *“Decent place, nothing special but the pizza was okay”* est classé **NÉGATIF à 97.2%** : dans le contexte des avis Yelp, *“nothing special”* et *“okay”* sont des signaux de déception modérée. Ceci est cohérent avec le dataset où 3–4 étoiles correspondent au label Positif : les clients *“ok”* mettent souvent 3 étoiles, donc Positif selon les labels, mais le modèle interprète l’insatisfaction implicite.

9 Bonus — Application Streamlit Déployée

Une application web interactive a été déployée sur **HuggingFace Spaces** avec le SDK Streamlit :

Application déployée en permanence sur **HuggingFace Spaces**

<https://huggingface.co/spaces/onsgaaya/sentimentYelpOns>

Déployé en permanence sur HuggingFace Spaces — accessible publiquement 24h/24

Fonctionnalité	Description
Zone de texte	Saisie libre d’un avis client
5 exemples rapides	Boutons pré-remplis (positif, négatif, mitigé)
Résultat coloré	Vert (POSITIF) / Rouge (NÉGATIF) + confiance
Graphique Plotly	Distribution des probabilités Nég/Pos
Tableau de bord	Comparaison des 3 modèles (accuracy, temps, GPU)
Décision ingénieur	Affichée dans la sidebar
Modèle utilisé	distilbert-base-uncased-finetuned-sst-2-english
Temps inférence	Affiché en ms pour chaque prédiction

Fichiers de déploiement HuggingFace Spaces

```
1 # requirements.txt
2 streamlit>=1.32.0
3 transformers>=4.46.0
4 torch>=2.0.0
5 plotly>=5.18.0
6 numpy>=1.24.0
7 pandas>=2.0.0
8
9 # Structure du depot HuggingFace
10 # app.py          <- application Streamlit principale
11 # requirements.txt
12
13 # Modele charge directement depuis HuggingFace Hub :
14 # distilbert-base-uncased-finetuned-sst-2-english
15 # (pas besoin d'uploader le modele - charge automatiquement
   )
```

10 Conclusion

10.1 Bilan du Projet

Ce projet a permis de construire un **pipeline complet d'analyse de sentiments**, de l'exploration des données brutes jusqu'au déploiement d'une application web interactive, en passant par l'implémentation et la comparaison de trois architectures de niveaux de complexité croissants.

Réponse à la Problématique

1. Un système automatique de classification de sentiments est **faisable avec une accuracy supérieure à 91%** sur les trois architectures.
2. Le meilleur compromis performance/coût est **TF-IDF + RL (92.58%)** pour un déploiement immédiat sans GPU.
3. Le résultat contre-intuitif (TF-IDF > DistilBERT sur 16K exemples) illustre le principe fondamental : **complexité $\propto$ volume de données**.
4. DistilBERT avec le dataset complet (560K) constitue la **V2 naturelle** et surpasserait probablement TF-IDF.

10.2 Synthèse des Résultats

Modèle	Accuracy	F1	Temps	GPU	Complexité	Recommandation
TF-IDF + RL	92.58%	0.93	19s	Non	Faible	Production immédiate
BiLSTM	85.72%	0.86	84s	Oui	Moyenne	V2 avec plus de données
DistilBERT	92.55%	0.92	216s	Oui	Élevée	V2 dataset complet

10.3 Perspectives et Améliorations

- **DistilBERT sur 560K avis** : entraînement sur le dataset complet avec un GPU A100 (disponible via Google Colab Pro).
- **LSTM avec embeddings pré-entraînés** : utiliser GloVe ou FastText pour initialiser les embeddings et améliorer le BiLSTM.
- **Analyse multi-classe** : passage de 2 à 5 classes (1–5 étoiles) pour une granularité plus fine.
- **Explicabilité** : visualisation des mots influents via SHAP ou LIME pour les prédictions TF-IDF (coefficients de la régression logistique).
- **API REST** : déploiement via FastAPI pour intégration dans un système CRM existant.

Mini-Projet IA — Ons GAAYA — GL4 A1 — Deep Learning & NLP — 2025/2026
Encadrante : Dr. Yosra HADJHASSEN — EPI Sousse, DIGITAL SCHOOL